

AWS Cost Intelligence System (OptiCloud)

This documentation submitted as required for the degree of Bachelor's in Computer and Information Sciences

By:

Ahmed Sameh Mohamed Saber [CSYS]

Hazem Ibrahim Mohamed [CSYS]

Ibrahim Mohamed Abdelsadek [CSYS]

John Ehab Fathy [CSYS]

Mahmoud Ahmed Kamel [CSYS]

Mariam Emad Fawzy [CSYS]

Under Supervision of:

Dr. Hafez Moawad

Computer System Department,
Faculty of Computer & Information Sciences,
Ain Shams University.

T.A. Mohamed Essam

Scientific Computing Department,
Faculty of Computer & Information Sciences,
Sciences,
Ain Shams University.

T.A. Seif Eldeen Sameh

Computer System Department,
Faculty of Computer & Information
Sciences,
Ain Shams University.

T.A. Radwa Osama

Computer System Department,
Faculty of Computer & Information Sciences,

July 2026

Ain Shams University
Faculty of Computer & Information Sciences
Computer System Department

Ain Shams University.

July 2026

Acknowledgments

All praise and thanks to Allah, the Most Merciful and Most Gracious, who granted us the strength, patience, and ability to complete this work. We humbly ask Him to accept this effort from us.

We would like to express our deepest gratitude to our families, whose unconditional love, encouragement, and support have been our foundation throughout all years of study. Words cannot express how much their sacrifices mean to us, and we hope that our success brings them pride and happiness.

We are sincerely thankful to our supervisors, **Dr. Hafez Moawad, T.A. Mohamed Essam, T.A. Seif Eldeen Sameh,** and **T.A. Radwa Osama** for their invaluable guidance, patience, and expertise throughout this project. Their continuous feedback and dedication played a major role in shaping this work.

We would also like to extend our appreciation to the Faculty of Computer & Information Sciences at Ain Shams University for providing us with the knowledge and environment to grow academically.

We acknowledge everyone who has contributed directly or indirectly to our success, including those who offered prayers, advice, or even a word of encouragement. Every small gesture has played a part in helping us reach this milestone.

Finally, we thank our friends and colleagues for their encouragement, motivation, and the memorable moments shared throughout this journey.

English Abstract

Cloud cost management has become a major challenge for organizations as cloud computing offerings grow in complexity. Although this challenge spans all major providers, this project focuses on Amazon Web Services (AWS) as the primary use case, given its dominant market position and the availability of rich datasets and APIs for experimentation. Many users incur unnecessary expenses caused by oversized resources, limited cost visibility, and the specialized expertise required for manual optimization. This project presents OptiCloud, an AWS Cost Intelligence System that provides cost forecasting and optimization using machine learning and artificial intelligence techniques. The system integrates data collection from AWS services, time-series forecasting models, anomaly detection, optimization algorithms, and an AI recommendation engine. Forecasting uses Seasonal Naive, ETS, and Prophet models, with model selection guided by data availability and forecast horizon. Cost optimization combines a binary linear programming solver with service-specific heuristic rules, while Gemini 2.5 Flash generates architecture recommendations for new users through a guided questionnaire. Experimental evaluation using simulated AWS environments and benchmark datasets achieved forecasting accuracy ranging from approximately 10% to 18% MAPE and identified potential monthly savings of about \$590 through 19 optimization recommendations. By unifying forecasting, anomaly detection, optimization, and AI-based guidance in a single interactive dashboard, the system reduces cloud waste, improves cost visibility, and helps both new and existing AWS users make more efficient infrastructure decisions.

Arabic Abstract

أصبحت إدارة تكاليف الحوسبة السحابية تحدياً رئيسياً للمؤسسات بسبب ازدياد تعقيد خدمات الحوسبة السحابية بشكل عام. وعلى الرغم من أن هذا التحدي موجود لدى جميع مزودي الخدمة الرئيسيين، إلا أن هذا المشروع يركز على خدمات أمازون ويب سيرفيسز (AWS) كحالة دراسية أساسية، نظراً لمكانتها السوقية المهيمنة وتوافر مجموعات البيانات والواجهات البرمجية اللازمة للتجربة. يعاني العديد من المستخدمين من نفقات غير ضرورية نتيجة استخدام موارد أكبر من الحاجة، وضعف الرؤية للتكاليف، والحاجة إلى خبرة متخصصة لإجراء عمليات التحسين يدوياً. يقدم هذا المشروع نظام OptiCloud، وهو نظام ذكي لإدارة وتحليل تكاليف AWS يعتمد على تقنيات الذكاء الاصطناعي وتعلم الآلة لتوفير التنبؤ بالتكاليف وتحسينها.

يتكون النظام من عدة وحدات تشمل جمع البيانات من خدمات AWS، ونماذج التنبؤ بالسلاسل الزمنية، وآليات اكتشاف الشذوذ، وخوارزميات التحسين، بالإضافة إلى محرك توصيات ذكي للمستخدمين الجدد. تم استخدام نماذج Seasonal Naive و ETS و Prophet مع آلية اختيار تلقائية للنموذج الأنسب وفقاً لحجم البيانات وفترة التنبؤ. كما تم تنفيذ التحسين باستخدام البرمجة الخطية الثنائية وقواعد مخصصة للخدمات المختلفة، بينما يوفر نموذج Gemini 2.5 Flash توصيات معمارية ذكية للمستخدمين الجدد من خلال استبيان موجّه. أظهرت النتائج دقة تنبؤ تتراوح بين 10% و 18% تقريباً، مع إمكانية تحقيق وفر شهري يقارب 590 دولاراً من خلال 19 توصية للتحسين. وقد أثبت النظام قدرته على دمج التنبؤ والتحسين واكتشاف الشذوذ والتوجيه الذكي ضمن لوحة تحكم موحدة وتفاعلية، مما يساهم في تقليل الهدر السحابي وتحسين كفاءة اتخاذ القرارات المتعلقة بالبنية التحتية لدى مستخدمي AWS الجدد والحاليين على حدّ سواء.

Table of Contents

Acknowledgments.....	1
English Abstract.....	2
Arabic Abstract.....	3
List of Figures.....	8
List of Tables.....	9
List of Abbreviations.....	10
CHAPTER 1 Introduction.....	12
1.1 Problem Definition.....	13
1.2 Motivation.....	14
1.3 Objective.....	14
1.4 Methodology.....	15
1.5 Time Plan.....	17
1.6 Thesis Outline.....	17
CHAPTER 2 Literature Review.....	19
2.1 Introduction.....	20
2.2 Cloud Cost Optimization.....	20
2.3 Time-Series Forecasting for Cloud Workloads.....	21
2.3.1 Classical Statistical Models.....	22
2.3.2 Modern Forecasting Frameworks.....	22
2.3.3 Benchmark Datasets.....	23
2.4 Anomaly Detection in Cloud Environments.....	24
2.4.1 Statistical Anomaly Detection.....	24
2.4.2 Machine Learning Approaches.....	25
2.5 AI-Based Architecture Recommendation Systems.....	25
2.5.1 Rule-Based and Expert Systems.....	25
2.5.2 Large Language Models for Architecture Guidance.....	26
2.5.3 Hybrid Optimization Systems.....	27
2.6 Summary and Research Gaps.....	27
CHAPTER 3 System Architecture & Methods.....	29
3.1 System Overview.....	30

3.2 Data Collection Layer.....	31
3.2.1 Real AWS Collection Pipeline.....	31
3.2.2 Demo Mode Data.....	32
3.3 Storage Layer.....	32
3.3.1 Database Design.....	32
3.3.2 Authentication and Security.....	33
3.4 ML Forecasting Engine.....	34
3.4.1 Data Preparation and Feature Engineering.....	34
3.4.2 Anomaly Detection Pre-processing.....	34
3.4.3 Forecasting Models.....	35
3.4.4 Model Selection Strategy.....	36
3.4.5 Model Evaluation.....	36
3.5 Cost Optimization Engine.....	37
3.5.1 Linear Programming Solver.....	37
3.5.2 Heuristic Rule Engine.....	38
3.5.3 Orchestration and Deduplication.....	39
3.6 AI Recommendation Module.....	40
3.6.1 Overview.....	40
3.6.2 Guided Questionnaire.....	40
3.6.3 Prompt Construction.....	40
3.6.4 Response Processing and Storage.....	41
3.7 Dashboard Architecture.....	41
3.8 Summary.....	42
CHAPTER 4 System Implementation & Results.....	44
4.1 Introduction.....	45
4.2 Development Environment.....	45
4.2.1 Software Stack.....	45
4.2.2 Project Structure.....	46
4.3 Datasets.....	47
4.3.1 Primary Dataset — Bitbrains GWA-T-12.....	47
4.3.2 Anomaly Detection Dataset — NAB CloudWatch Metrics.....	47
4.3.3 Supporting Datasets.....	47
4.3.4 Synthetic Demo Environment.....	48
4.4 Experiment Setup.....	49
4.4.1 Forecasting Evaluation Setup.....	49
4.4.2 Anomaly Detection Setup.....	50
4.4.3 Optimizer Setup.....	50

4.5 Forecasting Results.....	50
4.5.1 Model Performance by Forecast Horizon.....	50
4.5.2 Model Performance by Training Data Size.....	51
4.6 Anomaly Detection Results.....	52
4.7 Cost Optimization Results.....	53
4.7.1 Overall Savings.....	53
4.7.2 LP Solver Behavior.....	54
4.7.3 Confidence Distribution.....	54
4.8 AI Recommendation Module Results.....	55
4.9 Testing Results.....	55
4.10 Implementation Challenges and Solutions.....	56
4.11 Summary.....	57
CHAPTER 5 Run the Application.....	59
5.1 Introduction.....	59
5.2 System Startup.....	60
5.2.1 Prerequisites.....	60
5.2.2 Installation and Launch.....	60
5.2.3 Authentication Gate.....	60
5.3 Demo Mode Walkthrough.....	61
5.3.1 Home Page.....	61
5.3.2 Costs Page.....	62
5.3.3 Forecasts Page.....	63
5.3.4 Recommendations Page.....	64
5.3.5 Settings Page.....	65
5.4 Real Mode Setup.....	66
5.4.1 Create an IAM Role in AWS.....	66
5.4.2 Connect AWS Account via Dashboard.....	67
5.4.3 Running Data Collection.....	68
5.5 Key User Workflows.....	68
5.6 Summary.....	69
CHAPTER 6 Conclusion and Future Work.....	70
6.1 Project Summary.....	70
6.1.1 Objectives Achieved.....	71
6.1.2 Key Technical Achievements.....	72
6.2 Key Findings.....	72
6.2.1 Forecasting Insights.....	72
6.2.2 Anomaly Detection Robustness.....	72

6.2.3 Optimization Coverage.....	73
6.3 Impact and Value.....	73
6.3.1 Cost Reduction.....	73
6.3.2 Time Savings.....	73
6.3.3 Accessibility.....	74
6.4 Limitations and Trade-offs.....	74
6.4.1 Current Limitations.....	74
6.4.2 Known Trade-offs.....	75
6.5 Future Work.....	75
6.5.1 Short-term Enhancements (1-3 months).....	75
6.5.2 Medium-term Extensions (3-6 months).....	76
6.5.3 Long-term Vision (6-12 months).....	77
6.6 Conclusion.....	78
References.....	79

List of Figures

Figure 1.1 Time Plan..... 17

Figure 3.1 System Architecture Diagram.....31

Figure 3.2 Model Selection Decision Flow (30-day default).....36

List of Tables

Table 2.1 Comparison of Forecasting Models.....	23
Table 2.2 Key functional gaps	28
Table 3.1 AWS Service Collectors	31
Table 3.2 Database Table Categories.....	33
Table 3.3 Heuristic Optimization Rules	38
Table 3.4 System Components Summary	42
Table 4.1 Software Stack	44
Table 4.2 Supporting Datasets.....	46
Table 4.3 Synthetic Demo Environment	47
Table 4.4 Model MAPE by Forecast Horizon	49
Table 4.5 Model MAPE by Training Data Size (30-day forecast)	50
Table 4.6 Anomaly Severity Classification	51
Table 4.7 Optimizer Results	52
Table 4.8 Test Suite Summary	54

List of Abbreviations

<i>Abbreviation</i>	<i>Full Meaning</i>
AWS	Amazon Web Services
ML	Machine Learning
AI	Artificial Intelligence
ETS	Exponential Smoothing State Space Model
LP	Linear Programming
MILP	Mixed Integer Linear Programming
IQR	Interquartile Range
EC2	Elastic Compute Cloud
RDS	Relational Database Service
EBS	Elastic Block Store
S3	Simple Storage Service
ELB	Elastic Load Balancer
NAT	Network Address Translation
SDK	Software Development Kit
API	Application Programming Interface
MAPE	Mean Absolute Percentage Error
JSON	JavaScript Object Notation
SQLite	Lightweight embedded SQL database engine
LLM	Large Language Model
UI	User Interface
IaaS	Infrastructure as a Service
ARIMA	AutoRegressive Integrated Moving Average
SARIMAX	Seasonal AutoRegressive Integrated Moving Average with eXogenous inputs
NAB	Numenta Anomaly Benchmark
LSTM	Long Short-Term Memory
SVM	Support Vector Machine
GWA	Grid Workloads Archive
VM	Virtual Machine
OOM	Out of Memory
CBC	Coin-or Branch and Cut
TPS	Transactions Per Second
CV	Cross-Validation
RCU	Read Capacity Units
WCU	Write Capacity Units
ARN	Amazon Resource Name
STS	Security Token Service
RMSE	Root Mean Square Error

MAE	Mean Absolute Error
SaaS	Software as a Service
CPU	Central Processing Unit
I/O	Input / Output
CSV	Comma-Separated Values
PDF	Portable Document Format
IAM	Identity and Access Management
CFO	Chief Financial Officer
CFN	CloudFormation
IaC	Infrastructure as Code
ITFM	IT Financial Management
GPU	Graphics Processing Unit
TPU	Tensor Processing Unit



CHAPTER 1

Introduction



1.1 Problem Definition

Cloud computing has become a cornerstone of modern software infrastructure, with Amazon Web Services (AWS) standing as the dominant provider offering over 200 distinct services. Despite its widespread adoption, managing cloud costs effectively remains a critical challenge for organizations of all sizes. According to reports by Flexera, nearly 30–35% of all cloud spending is wasted due to idle resources, oversized instances, and suboptimal pricing model selections. While cloud cost waste is a universal challenge across all major providers including Microsoft Azure and Google Cloud Platform, this project focuses on AWS as the primary use case, given its dominant market position and the availability of rich open-source datasets and APIs for experimentation.

This problem manifests differently across user categories. New AWS users struggle to choose the right instance types and pricing plans, often lacking the in-house expertise needed to navigate the complexity of AWS offerings. Existing users, on the other hand, tend to overspend on idle or oversized resources without realizing it, as industry surveys report that 63% of small-to-medium companies lack adequate cost visibility. Additionally, manual optimization requires deep AWS expertise that most teams do not possess, making it impractical to manually review and optimize usage across hundreds of services.

The core issue is therefore twofold: the absence of intelligent automation for cost optimization, and the lack of accessible tools that serve both new and experienced AWS users within a unified system.

1.2 Motivation

The growing complexity of AWS pricing models and the unpredictability of cloud bills have created a pressing need for intelligent, automated cost management tools. Current solutions are either too generic, requiring significant manual effort, or too narrow in scope, targeting only specific services or user types. There is no single tool that combines cost forecasting, anomaly detection, resource optimization, and AI-based architecture guidance into one interactive platform accessible to non-experts.

OptiCloud is motivated by the need to bridge this gap. By applying machine learning and artificial intelligence, the system aims to deliver actionable, automated cost-saving recommendations directly from cloud usage data, with AWS serving as the primary implementation platform, without requiring specialized cloud expertise from the user. Handling multi-account AWS environments and complex pricing strategies, areas that typically require dedicated cloud engineers, is a core design goal of this project.

1.3 Objective

The primary objectives of the AWS Cost Intelligence System (OptiCloud) are as follows:

- **Cost Forecasting:** Provide accurate time-series forecasting of AWS costs using multiple machine learning models (Naive, Seasonal Naive, ETS, Prophet, and SARIMAX), designed to guide model selection based on available training data and the desired forecast horizon.

- **Anomaly Detection:** Identify unusual cost spikes and abnormal usage patterns using a combined Z-Score and IQR detection approach, enabling users to act before costs escalate.
 - **Cost Optimization:** Generate concrete cost-saving recommendations through a Binary Linear Programming solver for compute rightsizing and a rule-based heuristic engine for service-specific optimizations across EC2, RDS, Lambda, EBS, S3, DynamoDB, NAT Gateways, and ELB.
 - **AI-Based Architecture Recommendations:** Assist new AWS users with no existing infrastructure by providing personalized architecture recommendations through a guided 9-question questionnaire powered by the Gemini 2.5 Flash language model.
 - **Unified Dashboard:** Integrate all components into a single interactive Streamlit dashboard with five pages: Home, Costs, Forecasts, Recommendations, and Settings.
 - **User Authentication & Multi-tenancy:** Support multiple users with isolated data through a secure registration and login system using PBKDF2-HMAC-SHA256 (260,000 iterations, per-user random salt) password hashing.
 - **Demo Mode:** Provide a demo experience using pre-loaded synthetic data based on open-source datasets, requiring no AWS credentials.
-

1.4 Methodology

The development of OptiCloud followed an agile methodology, with work divided into iterative phases aligned with the project's seminar milestones. The system operates in two modes: **Demo Mode**, which uses pre-loaded synthetic data requiring no AWS credentials, and **Real Mode**, where users register an account, connect their AWS account via an IAM Role ARN, and the system is

designed to collect 12 months of historical data across all enabled regions. Both modes read from the same SQLite database through identical storage interfaces, so all downstream modules work identically regardless of the data source.

The system was built as a modular pipeline consisting of eight core components:

1. **Data Collection Layer** — Eleven service-specific collectors fetch usage metrics and cost data from AWS via the boto3 SDK and CloudWatch API, with built-in retry logic and rate-limit handling.
2. **Storage Layer** — A SQLite database with 30 tables stores all collected data, forecasts, anomalies, recommendations, and user settings.
3. **ML Forecasting Engine** — Five forecasting models are implemented with offline walk-forward cross-validation (120-day initial training window, 30-day step) to evaluate and select the best model per forecast horizon.
4. **Anomaly Detection Module** — A union of Z-Score (rolling 30-day window, threshold = 3.0 standard deviations) and IQR (multiplier = 1.5) methods flags both point spikes and persistent distributional outliers.
5. **Cost Optimization Engine** — A Binary LP solver (PuLP/CBC) handles compute rightsizing, while eight service-specific heuristic rules handle threshold-based decisions.
6. **AI Recommendation Module** — Gemini 2.5 Flash generates structured JSON architecture recommendations for new users based on nine guided onboarding questions.
7. **Testing** — 217 unit and integration tests ensure correctness across all modules.
8. **Dashboard** — A five-page Streamlit interface provides all functionality in an accessible, interactive format.

1.5 Time Plan

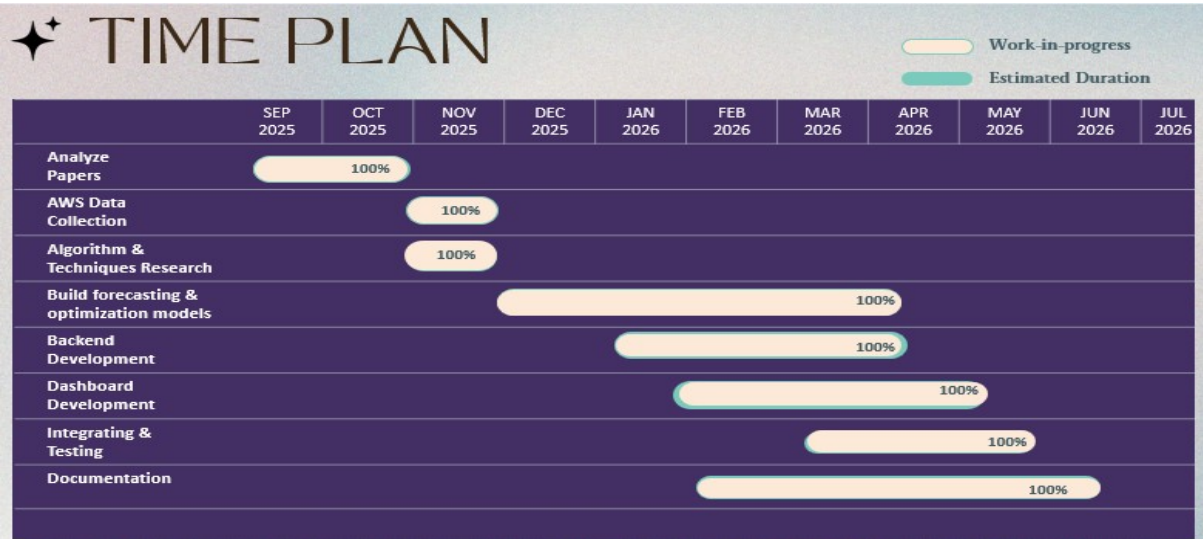


Figure 1.1 Time Plan

1.6 Thesis Outline

This documentation is organized into several chapters that guide the reader through the development process, from initial problem analysis to final implementation and evaluation. Each chapter builds upon the previous one to provide a comprehensive understanding of the AWS Cost Intelligence System (OptiCloud).

- **Chapter 1: Introduction**
Introduces the motivation behind the project, defines the cloud cost management problem, outlines the system's objectives, and presents the overall structure of the documentation.
- **Chapter 2: Literature Review**
Summarizes existing research on cloud cost optimization, time-series forecasting for cloud workloads, anomaly detection techniques, and AI-based resource recommendation systems. It highlights gaps in current

approaches and provides the foundation for the methods adopted in this project.

- **Chapter 3: System Architecture and Methods**

Describes the overall architecture of OptiCloud, including the data collection pipeline, storage layer, ML forecasting engine, anomaly detection module, cost optimization engine, and AI recommendation module. It explains the algorithms and design decisions behind each component.

- **Chapter 4: System Implementation and Results**

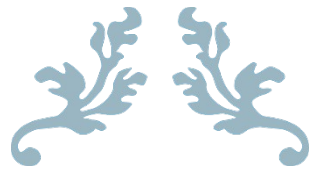
Explains the technical implementation of the system, including the datasets used, software tools and libraries, experiment setup, walk-forward cross-validation methodology, and the experimental results achieved across all modules.

- **Chapter 5: Run the Application**

Presents a walkthrough of the OptiCloud dashboard, demonstrating the five interactive pages — Home, Costs, Forecasts, Recommendations, and Settings — and illustrating how users interact with the system.

- **Chapter 6: Conclusion and Future Work**

Summarizes the project outcomes, discusses the impact of the solution on cloud cost management, and proposes future extensions including Infrastructure-as-Code export, a fine-tuned open-source LLM, and a savings validation feedback loop.



CHAPTER 2

Literature Review



2.1 Introduction

The growing adoption of cloud computing has introduced significant challenges in cost management, resource utilization, and infrastructure optimization. This chapter reviews existing research and related work across four main areas that form the foundation of the AWS Cost Intelligence System: cloud cost optimization, time-series forecasting for cloud workloads, anomaly detection techniques, and AI-based architecture recommendation systems. For each area, we identify the gaps that motivated the design decisions of OptiCloud.

2.2 Cloud Cost Optimization

Cloud cost optimization has been an active area of research since the widespread adoption of Infrastructure-as-a-Service (IaaS) platforms. The core challenge is matching allocated resources to actual workload demand, as overprovisioning wastes money while underprovisioning risks performance degradation.

Cortez et al. (2017) analyzed over 2 million virtual machines from Microsoft Azure's public dataset, demonstrating that the majority of cloud VMs are significantly overprovisioned, with average CPU utilization below 20% across most workload categories. Their findings justified the use of utilization-based rightsizing as a primary optimization strategy, which directly motivates OptiCloud's LP-based compute optimizer that uses P95 CPU utilization as the rightsizing criterion rather than average or peak values.

Shen et al. (2015) studied business-critical workloads hosted in cloud datacenters using the Bitbrains GWA-T-12 dataset, which covers performance metrics from 1,750 virtual machines in a managed hosting datacenter. Their statistical characterization of enterprise workload patterns revealed strong daily and weekly seasonality in CPU, memory, and network utilization. These findings influenced OptiCloud's data generation and ML model design, particularly the choice of weekly seasonality period and the use of P95 rather than maximum CPU for rightsizing to avoid over-engineering for rare spikes.

Industry reports from Flexera consistently estimate that 30–35% of all cloud spending is wasted due to idle resources, oversized instances, and suboptimal pricing model selections. Industry surveys further report that 63% of small-to-medium enterprises lack adequate cloud cost visibility, reinforcing the need for automated, accessible optimization tools that do not require deep AWS expertise.

Existing commercial tools such as AWS Cost Explorer, CloudHealth, and Spot.io address parts of this problem but are either too narrowly scoped, require significant manual configuration, or target enterprise customers with dedicated cloud engineering teams. OptiCloud differs by combining forecasting, anomaly detection, rightsizing, and AI-based guidance into a single unified system accessible to both new and experienced AWS users.

2.3 Time-Series Forecasting for Cloud Workloads

Accurate forecasting of cloud costs and resource utilization is essential for proactive cost management. A wide range of time-series forecasting methods have been applied to cloud workload prediction, ranging from classical statistical models to modern deep learning approaches.

2.3.1 Classical Statistical Models

Exponential Smoothing State Space Models (ETS) have long been used for time-series forecasting due to their ability to capture trend and seasonality with relatively little training data. Hyndman et al. demonstrated that ETS models provide competitive accuracy across a wide range of forecasting benchmarks while remaining computationally efficient, making them suitable for production systems where forecasts must be regenerated frequently. OptiCloud's evaluation confirmed this finding: ETS achieved the best MAPE of approximately 10–18% for 30-day cost forecasting when at least two months of historical data were available.

Seasonal AutoRegressive Integrated Moving Average with eXogenous inputs (SARIMAX) models extend classical ARIMA with seasonal components and external regressors. While SARIMAX can achieve high accuracy on stationary time series with well-defined seasonality, its auto-tuning via `pmdarima`'s `auto_arima` parameter search is computationally expensive, taking over 10 minutes per cross-validation fold in OptiCloud's experiments. This makes it impractical for interactive dashboard use, and it was therefore excluded from the default model selection pipeline.

2.3.2 Modern Forecasting Frameworks

Taylor and Letham (2018) introduced Prophet, an additive decomposition model developed at Facebook that handles daily, weekly, and yearly seasonality as well as holiday effects. In OptiCloud's walk-forward cross-validation, however, Prophet was among the least accurate of the evaluated models, with MAPE in the range of roughly 14–22%. Prophet also requires a minimum of six months of historical data to perform reliably, making it unsuitable for new users or accounts with limited history. OptiCloud is therefore designed to guide model selection by data availability, selecting Seasonal Naive when less than two months of data is available and ETS for horizons beyond 30 days.

Liu et al. (2025) conducted a comprehensive survey of public cloud datasets in ACM Computing Surveys, covering over 50 publicly available datasets from major cloud providers. Their analysis highlighted the importance of using real workload traces rather than purely synthetic data for model evaluation, which motivated OptiCloud's use of the Bitbrains dataset and the NAB CloudWatch metrics as primary data sources alongside generated synthetic data.

MODEL	BEST HORIZON	MIN DATA NEEDED	MAPE RANGE
<i>NAIVE</i>	Fallback only	Any	~25–40%
<i>SEASONAL NAIVE</i>	< 2 months data	2 weeks	~10–14%
<i>ETS</i>	30–240 days	2 months	~10–18%
<i>PROPHET</i>	7–30 days	6 months	~14–22%
<i>SARIMAX</i>	Any	6 months	Too slow for production

Table 2.1 summarizes the performance of all five forecasting models

2.3.3 Benchmark Datasets

The Numenta Anomaly Benchmark (NAB), introduced by Lavin and Ahmad (2015), provides over 50 labeled time-series files including actual AWS CloudWatch metrics with expert-labeled anomaly windows. NAB is the only publicly available dataset with labeled AWS-specific anomalies, making it a valuable public reference benchmark for anomaly-detection methods.

Additionally, the Cloud Workload Job Traces dataset from Kaggle provided 3,562 job records from a distributed cloud environment used to benchmark the forecasting engine on a different workload distribution than Bitbrains.

OptiCloud's walk-forward cross-validation methodology uses a 120-day initial training window with a 30-day step, an approach consistent with best practices identified in the time-series forecasting literature for avoiding data leakage while producing reliable out-of-sample error estimates.

2.4 Anomaly Detection in Cloud Environments

Cost anomaly detection is a critical component of cloud cost management, enabling organizations to identify unexpected spending spikes before they result in significant overruns.

2.4.1 Statistical Anomaly Detection

The Z-Score method detects anomalies by measuring how many standard deviations a data point falls from the rolling mean. While simple, rolling Z-Score with a sufficiently large window (30 days in OptiCloud) is effective at detecting sudden point spikes in cloud cost time series. Its main limitation is sensitivity to the choice of window size and threshold, and it can miss gradual distributional shifts.

The Interquartile Range (IQR) method is a non-parametric alternative that flags values falling beyond 1.5 times the IQR from the first and third quartiles.

Unlike Z-Score, IQR is robust to outliers in the training window itself, making it complementary rather than redundant. OptiCloud combines both methods using a union approach, flagging a data point as anomalous if either method detects it, which maximizes recall at the cost of slightly higher false positive rates.

2.4.2 Machine Learning Approaches

More advanced anomaly detection approaches include Isolation Forest, LSTM autoencoders, and One-Class SVM, which can detect complex multivariate anomalies across multiple services simultaneously. While these methods can achieve higher precision on labeled benchmarks, they require substantially more training data and computational resources than statistical methods, and their decisions are harder to explain to end users. Given that OptiCloud targets users without deep ML expertise, the interpretability and low data requirements of the Z-Score and IQR combination were prioritized over marginal accuracy gains from more complex models.

The NAB benchmark and the Cloud Resource Usage for Anomaly Detection dataset from Kaggle, which contains 1,440 rows of timestamped cloud metrics with anomaly labels covering workload types including Web Service, Backup, and Crypto Mining, informed the design of OptiCloud's anomaly detection module.

2.5 AI-Based Architecture Recommendation Systems

Recommending appropriate cloud infrastructure configurations to new users is a relatively recent research area, driven by the growing complexity of cloud service catalogs and the difficulty of translating business requirements into technical specifications.

2.5.1 Rule-Based and Expert Systems

Early approaches to cloud architecture recommendation relied on rule-based expert systems that mapped application characteristics to predefined architectural patterns. While effective for simple use cases, these systems

struggle to handle the combinatorial complexity of modern cloud offerings, where a single application might be deployed across dozens of valid configurations with significantly different cost and performance trade-offs.

2.5.2 Large Language Models for Architecture Guidance

The emergence of large language models (LLMs) has opened new possibilities for architecture recommendation. Models such as GPT-4 and Google Gemini can reason about infrastructure requirements expressed in natural language and generate structured recommendations that consider cost, performance, availability, and compliance constraints simultaneously.

OptiCloud uses Google Gemini 2.5 Flash for AI-based architecture recommendations, chosen for its fast inference speed, cost efficiency, and strong structured JSON output capabilities. The system guides new users through nine onboarding questions covering workload type, expected user count, uptime requirements, traffic patterns, availability requirements, monthly budget, AWS experience level, and additional constraints. These answers are converted into a structured prompt that instructs the model to prefer serverless and Graviton-based instance families, use modern instance generations, and match recommendation complexity to the user's stated experience level. The model returns a structured JSON response containing the recommended setup, estimated monthly cost, and a detailed explanation, which is then stored in the database and displayed on the dashboard.

The use of temperature = 0.3 reduces randomness in the model's output, producing more deterministic and comparable recommendations across runs while retaining enough flexibility to handle diverse user requirements.

2.5.3 Hybrid Optimization Systems

Hybrid systems that combine rule-based heuristics with ML-based optimization for cloud cost management have also been proposed. OptiCloud adopts a similar hybrid approach: the LP solver handles the mathematically well-defined rightsizing problem where a catalog of alternatives with quantifiable specifications exists, while the rule engine handles threshold-based decisions for services like S3, DynamoDB, and NAT Gateways where no continuous alternative menu exists. This division of responsibilities produces more transparent and explainable recommendations than a purely ML-based approach.

2.6 Summary and Research Gaps

The literature review reveals the following gaps that OptiCloud addresses:

- **No unified system:** Existing tools address forecasting, anomaly detection, or optimization in isolation. No single open-source tool combines all four capabilities — forecasting, anomaly detection, rightsizing optimization, and AI-based new-user guidance — into one interactive platform.
- **New user problem:** Existing optimization tools assume the user already has AWS infrastructure and historical data. No existing system provides meaningful guidance to users who are starting from scratch, which OptiCloud addresses through its AI recommendation module.
- **Data accessibility barrier:** Most optimization tools require deep AWS expertise to configure and interpret. OptiCloud's demo mode, guided questionnaire, and plain-language recommendation reasoning lower this barrier significantly.

- **Model selection gap:** Most forecasting implementations use a single fixed model regardless of data availability. OptiCloud's design to guide model selection by data availability — Seasonal Naive for less than two months of data, ETS for longer horizons — is intended to address the degraded performance of complex models like Prophet when training data is scarce.
- **Multi-account isolation:** Most open-source cloud cost tools are single-account. OptiCloud supports multi-account AWS environments through IAM Role assumption, with full data isolation per user through a user_id-keyed database schema.

Table 2.2 highlights the key functional gaps

Tool	Forecasting	Anomaly Detection	Rightsizing	New User Guidance
AWS Cost Explorer	Yes	Yes	No	No
Cloud Health	No	Yes	Yes	No
Spot.io	No	No	Yes	No
OptiCloud	Yes	Yes	Yes	Yes



CHAPTER 3

System Architecture & Methods



3.1 System Overview

OptiCloud is designed as a modular, multi-tenant AWS cost intelligence platform built around a single SQLite database as the central data store. The system operates in two modes: **Demo Mode**, which uses pre-loaded synthetic data requiring no AWS credentials, and **Real Mode**, where users register an account, connect their AWS account via an IAM Role ARN, and the system collects 12 months of historical data across all enabled regions. Both modes read from the same SQLite database through identical storage interfaces, ensuring all downstream modules work identically regardless of the data source. Although the system architecture and optimization methodology are designed to be extensible to other cloud providers, the current implementation targets AWS as the primary platform.

The overall architecture is divided into four layers as shown in Figure 3.1:

1. **Data Sources Layer** — Real AWS accounts or pre-loaded synthetic data
2. **Storage Layer** — SQLite database gateway with 30 tables
3. **Processing Layer** — ML engine, AI module, and cost optimizer
4. **Presentation Layer** — Streamlit dashboard with authentication gate

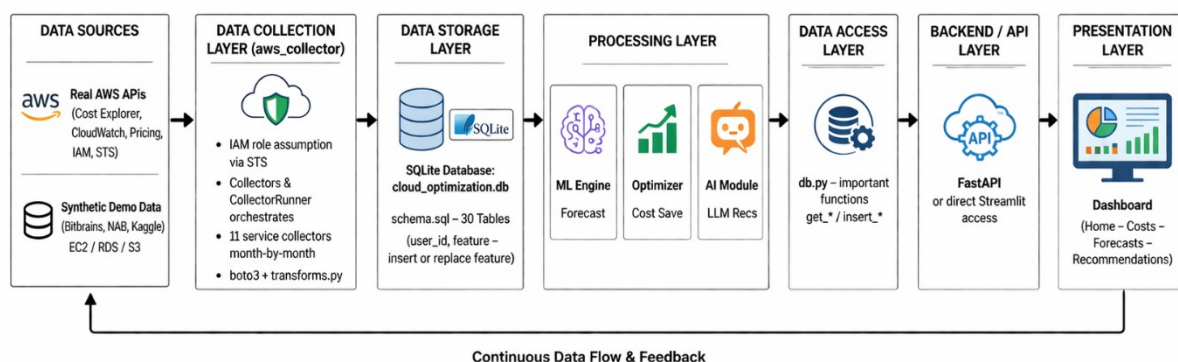


Figure 3.1 System Architecture Diagram

3.2 Data Collection Layer

3.2.1 Real AWS Collection Pipeline

The data collection layer consists of eleven service-specific collector classes, each inheriting from a common `BaseCollector` and implementing three core methods: `list_resources()`, `get_metrics()`, and `collect()`. The `CollectorRunner` orchestrates all collectors month-by-month, processing the last 12 months of historical data sequentially from oldest to newest.

The eleven collectors and their responsibilities are summarized here:

COLLECTOR	SERVICE	DATA COLLECTED
COSTCOLLECTOR	Cost Explorer	Daily costs, service costs, anomalies
EC2COLLECTOR	EC2 + EBS	Instances, volumes, CPU, network, disk metrics
RDSCOLLECTOR	RDS	DB instances, CPU, connections, storage metrics
LAMBDA COLLECTOR and the system calls sts:AssumeRole to obtain temporary credentials that are	Lambda	Functions, invocations, duration, error metrics
S3COLLECTOR	S3	Buckets, size, object count metrics
DYNAMODB COLLECTOR database. Each AWS API call is wrapped in a try/except block so that a single failed metric call does not stop the entire collection pipeline: the retry count and	DynamoDB	Tables, read/write capacity metrics
ELASTICACHECOLLECTOR	ElastiCache	Clusters, CPU, memory, cache hit metrics
ECSCOLLECTOR timeout.	ECS/Fargate	Services, CPU, memory, task count metrics
NATGATEWAYCOLLECTOR	NAT Gateway	Gateways, bytes, packets, connection metrics
ELBCOLLECTOR The database ships with pre-loaded synthetic data originally generated from	ALB/NLB	Load balancers, requests, response time metrics
PRICINGCOLLECTOR	Table 3.1 AWS Service Pricing API	EC2, RDS, Lambda, S3 pricing data

memory utilization patterns, AWS CloudWatch metrics for anomaly patterns,

and the Kaggle AWS Pricing dataset for pricing data. The synthetic environment simulates a mid-size SaaS startup with a monthly bill of approximately \$1,500–\$2,500, covering 8 EC2 instances, 2 RDS instances, 3 ElastiCache nodes, 4 ECS services, 4 Lambda functions, 3 DynamoDB tables, 4 S3 buckets, 8 EBS volumes, 2 NAT Gateways, and 3 ELB instances across 365 days of cost history.

3.3 Storage Layer

3.3.1 Database Design

All data is stored in a single SQLite database (data/cloud_optimizer.db) managed by storage/db.py. The database contains 30 tables organized into five categories as shown:

SQLite was chosen as the data store for four reasons: it provides ACID transactions and indexed queries without requiring an external database server; it supports multi-user isolation through a `user_id` foreign key on every table; its Write-Ahead Logging (WAL) mode enables concurrent reads from the

Table 3.2 Database Table Categories

CATEGORY	TABLES	COUNT
CORE	users, aws_connections	2
COST DATA	daily_costs, service_costs, service_region_costs	3
METRICS	ec2, rds, elasticache, ecs, lambda, dynamodb, ebs, s3, nat_gateway, elb metrics	10
INVENTORY	ec2, rds, elasticache, ecs, lambda, ebs, s3, dynamodb, nat_gateway, elb instances	10
OUTPUT	forecasts, recommendations, anomalies, ai_recommendations, instance_pricing	5

dashboard while the collector writes; and all insert functions use `INSERT OR REPLACE` on primary keys for automatic deduplication, meaning the collector can be safely re-run without creating duplicate rows.

3.3.2 Authentication and Security

User authentication is implemented directly in the storage layer. Passwords are hashed using PBKDF2-HMAC-SHA256 with a per-user random 16-byte salt and 260,000 iterations, stored as a salt:hash string. The `register_user()` function creates a new user with a `usr-` prefixed 12-character hex identifier and raises a

ValueError on duplicate email addresses. The `authenticate_user()` function verifies credentials and updates the `last_login_at` timestamp on successful login. AWS connections are stored in the `aws_connections` table, which maps users to their AWS accounts via IAM Role ARNs. The `delete_aws_connection()` function is scoped to the requesting user, preventing users from deleting other users' connections.

3.4 ML Forecasting Engine

3.4.1 Data Preparation and Feature Engineering

Before training, cost data is loaded from the `daily_costs` table and enriched with time-based features including hour, day of week, month, weekend indicator, and sine/cosine cyclical encodings. Lag features are computed at 1-day, 7-day, and 30-day intervals to capture short-term, weekly, and monthly patterns respectively.

3.4.2 Anomaly Detection Pre-processing

The system is designed to detect anomalies and exclude them from training data before any forecasting model is fitted. This pre-processing step prevents cost spikes from distorting model parameters and is implemented using two complementary statistical methods combined through a union approach:

Z-Score Detection: A rolling 30-day window computes the mean and standard deviation of recent costs. Any data point falling more than 3.0 standard deviations from the rolling mean is flagged as a point spike anomaly. This method is fast and effective at detecting sudden cost jumps.

IQR Detection: The Interquartile Range method computes the first quartile (Q1) and third quartile (Q3) of the cost distribution. Any value falling below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is flagged as a distributional outlier. Unlike

Z-Score, IQR is robust to non-normal distributions and catches persistent gradual shifts that Z-Score may miss.

The union of both methods maximizes recall, so that flagged cost anomalies can be excluded from training data even when only one method detects them.

Detected anomalies are also stored in the anomalies table with severity levels of info, warning, or critical based on the percentage deviation from expected cost.

3.4.3 Forecasting Models

Five forecasting models are implemented with a unified `fit(df) / predict(horizon)` interface, all returning a DataFrame with columns [date, forecast, lower, upper]:

Naive Forecaster: Repeats the last observed value for all future dates. Serves as a performance baseline.

Seasonal Naive Forecaster: Uses the value from exactly 7 days ago for each future date, capturing the weekly seasonality present in cloud costs where weekday spending is typically 30% higher than weekends.

ETS Forecaster: Exponential Smoothing State Space Model implemented using statsmodels. Captures trend and seasonality with additive or multiplicative components, automatically falling back to a non-seasonal variant when less than two weeks of data is available.

Prophet Forecaster: Facebook's additive decomposition model with weekly and yearly seasonality components. Intended for short horizons of 7–30 days, but in walk-forward cross-validation it ranked among the least accurate of the evaluated models (MAPE ~14–22%); an illustrative training-size analysis (Table 4.5) further shows its MAPE exceeding 1,000% on very small training sets.

SARIMAX Forecaster: Auto-tuned Seasonal ARIMA with eXogenous inputs using pmdarima's `auto_arima` parameter search. While capable of high

accuracy, its parameter search takes over 10 minutes per cross-validation fold, making it impractical for interactive dashboard use.

3.4.4 Model Selection Strategy

OptiCloud is designed to use a model selection strategy based on the amount of available historical data and the requested forecast horizon, as illustrated in Figure 3.2.

For **30-day forecasting** (the default horizon):

- Less than 2 weeks of data → **Naive** (fallback)
- 2 weeks to 2 months of data → **Seasonal Naive** (MAPE: ~10–14%)
- 2 months or more of data → **ETS** (MAPE: ~10–18%)

For **short-horizon forecasting** (7–30 days) with sufficient data (6+ months):

- **Seasonal Naive and ETS** achieve the best accuracy (MAPE: ~10–12%)

For **long-horizon forecasting** (60–240 days):

- **ETS** is the most reliable choice (MAPE: ~11–12%)

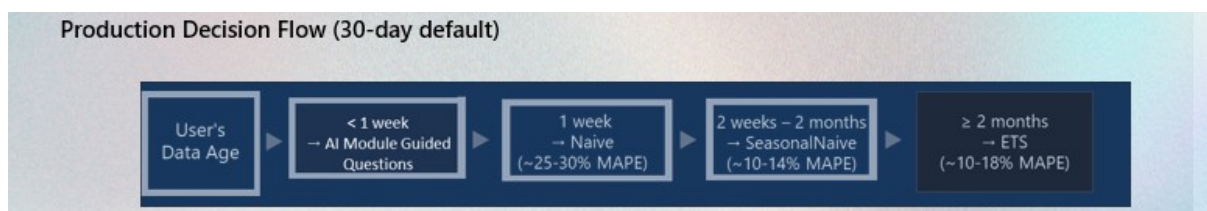


Figure 3.2 Model Selection Decision Flow (30-day default)

3.4.5 Model Evaluation

All models are evaluated offline using walk-forward cross-validation with an expanding training window. The evaluation parameters used in this study are: 120-day initial training window, 30-day step size, and MAPE as the primary evaluation metric. This approach avoids data leakage by ensuring the model never trains on data that comes after the validation period, producing reliable out-of-sample error estimates consistent with production conditions.

3.5 Cost Optimization Engine

The cost optimization engine uses two complementary approaches — a Linear Programming solver for compute resources and a rule-based heuristic engine for all other services — based on the structural difference between the two problem types.

3.5.1 Linear Programming Solver

The LP solver handles EC2 and RDS rightsizing, formulated as a Binary Mixed Integer Linear Programming (MILP) problem using the PuLP library with the CBC backend:

Decision Variables: Binary variable $x[\text{instance}, \text{candidate}] \in \{0, 1\}$, where 1 means assigning instance to candidate type.

Objective Function: Minimize the total monthly cost across all instances and their assigned candidate types.

Constraints:

- Assignment: Each instance must be assigned exactly one candidate type
- CPU: The assigned type must provide $\text{vCPUs} \geq \text{P95 CPU utilization} \times \text{current vCPUs} \times 1.3$ headroom factor
- Memory: The assigned type must provide $\text{memory} \geq \text{P95 memory utilization}$ (EC2 only; skipped for RDS due to metric unreliability)
- Budget: Total cost must not exceed the configured budget cap (optional)

The $1.3\times$ headroom factor provides a 30% buffer above the P95 utilization to accommodate unexpected traffic spikes without performance degradation. P95 is used rather than average (which hides spikes) or maximum (which represents rare outliers).

The solver handles three edge cases gracefully: if the LP is infeasible (no candidate type can satisfy the requirements), the instance is skipped; if the

optimal type is the current type, no recommendation is generated; and if the optimal type costs more than the current type, no recommendation is generated. For RDS instances, memory constraints are skipped because CloudWatch reports FreeableMemory in raw bytes rather than utilization percentage, and database engines reserve 25–75% of RAM at zero load, making the raw metric unreliable for rightsizing decisions.

3.5.2 Heuristic Rule Engine

Eight rule-based checks handle threshold-based optimization decisions where no continuous menu of alternatives exists:

The engine includes safety checks to prevent harmful recommendations:
 Lambda memory is not reduced if the average plus two standard deviations would exceed the new tier (preventing Out-of-Memory errors on traffic spikes);
 EBS idle detection requires at least 7 days of hourly metrics; and ELB deletion requires both zero targets in inventory AND zero traffic in metrics.

3.5.3 Orchestration and Deduplication

Table 3.3 Heuristic Optimization Rules

RULE	SERVICE	TRIGGER CONDITION	RECOMMENDATION
EC2 PRICING	EC2	On-demand + running 60+ days	Switch to Reserved 1-year
RDS PRICING	RDS	On-demand + running 60+ days	Switch to Reserved 1-year
LAMBDA MEMORY	Lambda	Avg memory used < 50% of allocation	Downsize to next lower memory tier
EBS VOLUMES	EBS	gp2 type, OR unattached, OR idle > 90%	Upgrade gp2 → gp3, or delete unused
S3 TIERING	S3	STANDARD class + < 100 daily requests	Switch to INTELLIGENT_TIERING
DYNAMODB MODE	DynamoDB	PROVISIONED + both RCU and WCU < 50% utilized	Switch to ON_DEMAND
NAT GATEWAY	VPC	Monthly cost > \$30	Add VPC Gateway Endpoints
ELB IDLE	ELB	Zero targets AND no traffic	Delete load balancer

The engine.py orchestrator runs both the LP solver and all rule checks, then deduplicates results by keeping only the highest-savings recommendation per (resource_id, recommendation_type) pair. This prevents duplicate entries when both the LP solver and a heuristic rule produce a recommendation for the same resource. All recommendations are written to the recommendations table and returned sorted by monthly savings in descending order.

3.6 AI Recommendation Module

3.6.1 Overview

The AI module addresses the cold-start problem for new users who have no existing AWS infrastructure or historical cost data. Rather than presenting a blank dashboard, OptiCloud guides new users through a 9-question onboarding questionnaire and generates personalized AWS architecture recommendations using Google Gemini 2.5 Flash.

3.6.2 Guided Questionnaire

The nine questions collect the following information from the user:

- Business type (web application, mobile backend, e-commerce, data analytics, internal tools)
- Expected daily active users
- Uptime requirement (business hours, 24/5, 24/7)
- Optimization priority (cost, performance, or balance)
- Traffic pattern (steady, predictable peaks, unpredictable spikes)
- Availability requirement (single-AZ, multi-AZ, highly available)
- Monthly budget
- AWS experience level (beginner, intermediate, advanced)
- Additional requirements (free text for compliance, region preferences, etc.)

3.6.3 Prompt Construction

User answers are converted into a structured prompt by `prompt_builder.py`. The prompt instructs Gemini 2.5 Flash to prefer serverless and Graviton-based

instance families (t4g, m7g, c7g, r7g), use modern instance generations, consider AWS Free Tier for tight budgets, match recommendation complexity to the user's stated experience level, and return a pure JSON response with no markdown formatting.

The required JSON output format contains three fields: `recommended_setup` (with sub-fields for compute, database, storage, networking, and other services), `estimated_cost` (monthly USD estimate), and `explanation` (detailed reasoning for each service choice).

3.6.4 Response Processing and Storage

The model is called with a temperature of 0.3 to reduce output randomness and produce more deterministic, comparable recommendations across runs. The `extract_json()` function handles cases where the model wraps its JSON response in markdown code blocks by extracting the substring between the first opening brace and the last closing brace.

All recommendations are stored in the `ai_recommendations` table, including the full prompt text, structured output, estimated cost, explanation, model identifier, and raw model response for debugging and future fine-tuning analysis.

3.7 Dashboard Architecture

The Streamlit dashboard is organized around an authentication gate that runs before any page is rendered. The `auth.py` module manages session state, login, registration, and demo mode access. Once authenticated, users access five pages

through a sidebar navigator with an account switcher for managing multiple AWS connections:

- **Home:** Overview metrics, 30-day cost trend, top services by cost, top 3 recommendations, and recent anomalies
 - **Costs:** Date range selector, daily cost line chart, service breakdown bar and pie charts, stacked area chart, and CSV export
 - **Forecasts:** Model selector, horizon selector (7, 14, 30, 60, or 90 days), automatic model training, forecast visualization with confidence intervals, and multi-model comparison mode
 - **Recommendations:** Summary metrics, multi-select filters by service/type/priority, sorting options, detailed recommendation cards with implementation guidance, and CSV export
 - **Settings:** User profile editing, AWS account connection management via IAM Role ARN, and system status display
-

3.8 Summary

This table summarizes all system components, the algorithms used, and the libraries that implement them.

COMPONENT	ALGORITHM / METHOD	LIBRARY	KEY METRIC
DATA COLLECTION	Service-specific collectors + STS role assumption	boto3	11 services, 12 months
STORAGE	SQLite with WAL, INSERT OR REPLACE upsert	sqlite3	30 tables, user_id isolation
ANOMALY DETECTION	Z-Score (window=30, $\sigma=3.0$) + IQR ($\times 1.5$) union	numpy/pandas	Union of both methods
FORECASTING (SHORT)	Prophet additive decomposition	prophet	~14–22% MAPE
FORECASTING (MEDIUM)	ETS Exponential Smoothing	statsmodels	10–18% MAPE
FORECASTING (FALLBACK)	Seasonal Naive (7-day pattern)	numpy	~10–14% MAPE
COMPUTE RIGHTSIZING	Binary MILP, P95 CPU + 1.3 $\times$ headroom	PuLP/CBC	\$570+ savings identified
SERVICE OPTIMIZATION	8 threshold-based heuristic rules	custom	13 additional recommendations
AI RECOMMENDATIONS	Gemini 2.5 Flash, temperature=0.3	google-genai	9-question onboarding
EVALUATION	Offline walk-forward CV (120d init, 30d step)	custom	MAPE primary metric
TESTING	Unit + integration tests	pytest	217 tests passing
DASHBOARD	Authentication gate + 5 pages	Streamlit	5 pages, multi-account

Table 3.4: System Components Summary



CHAPTER 4

System Implementation & Results



4.1 Introduction

This chapter describes the technical implementation of OptiCloud, covering the software environment, datasets used, experiment setup, and the experimental results achieved across all system modules. The chapter is organized to follow the system pipeline: data collection and storage, ML forecasting evaluation, anomaly detection validation, cost optimization results, and AI recommendation module testing.

4.2 Development Environment

4.2.1 Software Stack

The system was developed entirely in Python 3.12 and built around a set of well-established open-source libraries. Table 4.1 summarizes the complete software stack used across all modules.

COMPONENT	TECHNOLOGY	VERSION	PURPOSE
LANGUAGE	Python	3.12	Core development language
DASHBOARD	Streamlit	Latest	Interactive web UI
AWS SDK	boto3	Latest	AWS API access
DATABASE	SQLite	Built-in	Data storage (30 tables)
ML	Statsmodels	Latest	ETS model
FORECASTING			
ML	Prophet	Latest	Prophet model
FORECASTING			
ML	Pmdarima	Latest	SARIMAX auto-tuning
FORECASTING			
OPTIMIZATION	PuLP	Latest	LP/MILP solver
LP BACKEND	CBC	Built-in	PuLP solver backend
AI MODULE	Google-genai	Latest	Gemini 2.5 Flash API
DATA	Pandas	Latest	DataFrame operations
PROCESSING			
NUMERICAL	Numpy	Latest	Array operations
VISUALIZATION	Plotly	Latest	Interactive charts
TESTING	Pytest	Latest	Unit and integration tests
PASSWORD	hashlib	Built-in	PBKDF2-HMAC-SHA256
HASHING			password hashing

Table 4.1 Software Stack

4.2.2 Project Structure

The project is organized into seven core modules, each with a clear single responsibility:

cloud-gp/

- |— aws_collector/ # 11 service collectors + CollectorRunner
- |— storage/ # SQLite gateway (30 tables, auth + data API)
- |— ml_engine/ # 5 forecasting models + anomaly detection
- |— ai_module/ # Gemini 2.5 Flash recommendations
- |— optimizer/ # LP solver + 8 heuristic rules
- |— dashboard/ # Streamlit UI (auth gate + 5 pages)
- |— tests/ # 217 unit and integration tests

4.3 Datasets

Three categories of data were used in the implementation and evaluation of OptiCloud.

4.3.1 Primary Dataset — Bitbrains GWA-T-12

The Bitbrains GWA-T-12 dataset is the primary source for VM utilization patterns. It contains performance metrics from 1,750 virtual machines in a managed hosting datacenter, including CPU utilization, memory usage, disk I/O, and network throughput at 5-minute intervals. This dataset was used to generate realistic CPU utilization profiles for the synthetic EC2 instances in the demo environment, including the distinct behavioral patterns for production web servers, API servers, batch processors, and development instances.

4.3.2 Anomaly Detection Dataset — NAB CloudWatch Metrics

The Numenta Anomaly Benchmark (NAB) provides over 50 labeled time-series files including actual AWS CloudWatch metrics with expert-labeled anomaly windows. It was reviewed as a public reference benchmark for anomaly-detection methods; the Z-Score and IQR module is designed to capture both point spikes and persistent distributional outliers of the kind such benchmarks contain.

4.3.3 Supporting Datasets

Three additional Kaggle datasets were used for specific validation tasks as summarized here:

DATASET	SOURCE	SIZE	PURPOSE
EC2 INSTANCE METRICS	sakthivelank, Kaggle	1 day	Pipeline format validation

CLOUD RESOURCE USAGE FOR ANOMALY DETECTION	programmer3, Kaggle	1440 rows	Second anomaly validation source
CLOUD WORKLOAD JOB TRACES	zoya77, Kaggle	3562 jobs	Forecasting benchmark on different distribution
AWS PRICING DATASET	justsahil, Kaggle	~400 MB	Pricing cross-validation

4.3.4 Synthetic Demo Environment

The pre-loaded demo database simulates a mid-size SaaS startup with a monthly AWS bill of approximately \$1,500–\$2,500. Table 4.3 describes the simulated environment.

SERVICE	COUNT	DETAILS
EC2 INSTANCES	8	2 prod-web, 1 prod-api, 1 prod-cache, 1 staging-web, 1 staging-api, 1 dev, 1 batch
RDS INSTANCES	2	prod-postgres (r5.large), staging-postgres (t4g.medium)
ELASTICACHE NODES	3	1 prod Redis, 1 prod Memcached, 1 staging Redis

ECS SERVICES	4	Fargate services across prod and staging clusters
LAMBDA FUNCTIONS	4	image-resizer, email-sender, log-processor, webhook-handler
DYNAMODB TABLES	3	On-Demand and Provisioned modes
S3 BUCKETS	4	uploads, logs, backups, static-assets
EBS VOLUMES	8	One per EC2 instance (gp2/gp3/io2)
NAT GATEWAYS	2	One per VPC
ELB INSTANCES	3	ALBs for prod and staging
COST HISTORY	365	~\$1,500–\$2,500/month

The synthetic cost data includes several realistic features: weekly seasonality with weekday costs 30% higher than weekends, an upward trend of +1.5% per month, month-end cost bumps of +8% in the last 3 days, five injected anomaly spikes at 2–3× normal cost, and Gaussian noise of $\pm 12\%$.

4.4 Experiment Setup

4.4.1 Forecasting Evaluation Setup

All forecasting models were evaluated using walk-forward cross-validation with the following parameters:

- **Dataset:** 365 days of daily cost data from the synthetic demo environment
- **Initial training window:** 120 days
- **Step size:** 30 days
- **Evaluation metric:** MAPE (Mean Absolute Percentage Error)
- **Secondary metrics:** RMSE (Root Mean Square Error) and MAE (Mean Absolute Error)

- **Models evaluated:** Naive, Seasonal Naive, ETS, Prophet, SARIMAX

Walk-forward cross-validation was chosen over standard train/test split because it respects the temporal ordering of time-series data and prevents data leakage, where future information could influence model training.

4.4.2 Anomaly Detection Setup

The anomaly detection module was configured as follows:

- **Z-Score:** Rolling window of 30 days, threshold of 3.0 standard deviations
- **IQR:** Multiplier of 1.5
- **Combination:** Union of both methods (a point is flagged if either method detects it)
- **Surge detection:** Five injected cost spikes in the synthetic data, all flagged by a global $\mu+2\sigma$ surge threshold ($\approx \$120.38/\text{day}$; maximum z-score 6.9)

4.4.3 Optimizer Setup

The cost optimization engine was run against the full synthetic demo environment:

- **LP solver:** PuLP with CBC backend
- **Budget cap:** \$5,000/month (default)
- **Headroom factor:** $1.3\times$ P95 CPU utilization
- **Minimum data requirement:** 60 days of metrics before generating pricing plan recommendations
- **Services covered:** EC2, RDS, Lambda, EBS, S3, DynamoDB, NAT Gateway, ELB

4.5 Forecasting Results

4.5.1 Model Performance by Forecast Horizon

Table 4.4 presents the MAPE results for all four main models across five of the seven forecast horizons evaluated using walk-forward cross-validation.

Key findings from the horizon analysis:

- **Prophet** is among the least accurate of the four models, with MAPE ranging from roughly 14% to 22% and never winning at any horizon
- **ETS** is the most accurate model overall ($\approx 11.2\%$ MAPE in cross-validation), winning every horizon from 30 to 90 days
- **Seasonal Naive** is the most accurate model at the shortest horizons (7 and 14 days, $\approx 10\text{--}11\%$ MAPE) due to the strong weekly seasonality pattern in cloud costs

Table 4.4: Model MAPE by Forecast Horizon

HORIZON	NAIVE	SEASONAL	ETS	PROPHET	WINNER
	NAIVE				
7 DAYS	25.16%	10.18%	11.28%	14.41%	Seasonal Naive
14 DAYS	27.90%	10.84%	11.24%	20.47%	Seasonal Naive
30 DAYS	27.26%	12.28%	11.20%	20.93%	ETS
60 DAYS	27.87%	12.66%	11.39%	19.70%	ETS
90 DAYS	27.38%	12.91%	12.15%	18.08%	ETS

- **Naive** performs poorly across all horizons, confirming it should only be used as a last-resort fallback

4.5.2 Model Performance by Training Data Size

A critical finding from the evaluation was the severe degradation of Prophet with limited training data. Table 4.5 shows the MAPE for a 30-day forecast horizon across different amounts of historical training data.

TRAINING DATA	SEASONAL NAIVE	ETS	PROPHET	WINNER
7 DAYS	14.0%	297%	1015%	Seasonal naive

14 DAYS	10.1%	22.8%	1176%	Seasonal naive
1 MONTHS	9.3%	12.9%	46.7%	Seasonal naive
2 MONTHS	10.3%	9.7%	1552%	ETS
6 MONTHS	14.6%	11.1%	16.8%	ETS
1 YEAR	16.0%	12.4%	14.9%	ETS
1.5 YEARS	12.4%	10.9%	12.2%	ETS

This analysis produced the most important engineering decision in the project: **Prophet should never be used for 30-day forecasting regardless of data availability**, as ETS consistently outperforms it even with 1.5 years of training data. These training-size figures are illustrative results from an earlier offline forecasting analysis rather than the canonical walk-forward cross-validation; in that canonical evaluation (Table 4.4) Prophet ranked among the least accurate models at every horizon, so it is not recommended as a default even for short-horizon forecasts.

4.6 Anomaly Detection Results

Five cost surges in the synthetic dataset were detected using a global $\mu+2\sigma$ threshold ($\approx \$120.38/\text{day}$; maximum z-score 6.9). The key characteristics of the detected anomalies are summarized here:

SEVERITY LEVEL	DEVIATION FROM EXPECTED	ACTION
INFO	< 50% above expected	Logged for monitoring
WARNING	50–100% above expected	Flagged on dashboard
CRITICAL	> 100% above expected	Highlighted with alert

Table 4.6: Anomaly Severity Classification

The union approach (flagging a point if either Z-Score or IQR detects it) is designed to favor recall, catching both point spikes and persistent outliers. The trade-off is a slightly higher false positive rate compared to using either method alone, which is acceptable given that the module is designed to surface anomalies for review and, by intent, to keep them out of forecasting training data rather than to minimize false alarms.

4.7 Cost Optimization Results

4.7.1 Overall Savings

Running the optimizer against the full synthetic demo environment produced 19 recommendations with a total identified monthly savings of **\$590.40**. Table 4.7 presents the complete breakdown.

4.7.2 LP Solver Behavior

The LP solver correctly handled one infeasible case: the batch-processor EC2 instance (c5.2xlarge running at ~90% CPU) required approximately 10.4 vCPUs with the 1.3× headroom factor, but the largest available candidate in the pricing catalog provides only 8 vCPUs. The solver correctly skipped this

Table 4.7: Optimizer Results

SERVICE	RESOURCE	RECOMMENDATION TYPE	MONTHLY SAVINGS
RDS	prod-postgres-primary	Rightsizing	\$327.04
RDS	staging-postgres	Rightsizing	\$35.77
EC2 (×6)	6 on-demand instances	Pricing plan switch	\$208.42
EBS (×6)	gp2 volumes + orphan + idle	Volume upgrade / delete	\$12.60
NAT (×2)	2 NAT gateways	Replace with VPC endpoints	\$5.67
S3	app-backups bucket	Storage class switch	\$0.79
LAMBDA	webhook-handler	Memory resize	\$0.11
TOTAL	19 recommendations		\$590.40/month

instance rather than generating an incorrect recommendation, demonstrating correct handling of this edge case.

4.7.3 Confidence Distribution

Of the 19 recommendations generated:

- **High confidence** (>20% savings or >180 days of data): RDS rightsizing, EC2 pricing plan switches
- **Medium confidence** (moderate savings with sufficient data): EBS volume upgrades
- **Low confidence** (estimated savings based on usage patterns): NAT Gateway endpoint replacement

4.8 AI Recommendation Module Results

The AI recommendation module was tested with 12 unit and integration tests, all passing. The module correctly handles the following scenarios:

- Successful API calls return a structured JSON response with `recommended_setup`, `estimated_cost`, and `explanation` fields
- Missing API key returns a structured error dictionary rather than raising an exception
- Invalid JSON responses from the model are handled gracefully through the `extract_json()` function
- Empty API responses are detected and reported as structured errors

As an illustrative example, a sample output for an e-commerce use case with a \$500–\$2,000 monthly budget showed a recommended architecture of ECS Fargate with auto-scaling, RDS PostgreSQL Multi-AZ, S3 with CloudFront CDN, and ElastiCache Redis, with an estimated monthly cost of \$875.25 produced by the language model (a non-deterministic illustration, not a measured result).

4.9 Testing Results

The complete test suite consists of 217 passing tests across ten test files, with one further suite (`test_synthetic.py`) that cannot be collected because the optional `data_generation` package is not shipped, as summarized here:

TEST FILE	MODULE	TESTS	STATUS
	TESTED		
TEST_CONFIG.PY	Root configuration	8	All passing

TEST_DATE_UTILS.PY	Data utilities	14	All passing
TEST_ML_UTILS.PY	ML engine	38	All passing
TEST_STORAGE.PY	Storage layer	24	All passing
TEST_SYNTHETIC.PY	Data generation	—	Collection error
TEST_OPTIMIZER.PY	Optimizer	27	All passing
TEST_AI_MODULE.PY	AI module	12	All passing
TEST_AUTH.PY	Authentication	20	All passing
TEST_AWS_CONFIG.PY	AWS config	7	All passing
TEST_COMPONENTS.PY	Dashboard components	25	All passing
TEST_TRANSFORMS.PY	Data transforms	42	All passing
TOTAL		217	217 passing (+1 collection error)

The test suite reports no failures; in particular, test_optimizer.py passes all 27 of its tests. The only collection issue is in test_synthetic.py, which cannot be imported because the optional data_generation package is not shipped, so its tests are skipped rather than executed and the production functionality is unaffected.

4.10 Implementation Challenges and Solutions

During implementation, four technical challenges were encountered and resolved. Table 4.9 summarizes these challenges and their solutions.

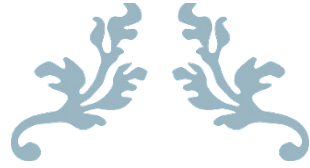
Table 4.9: Implementation Challenges and Solutions

CHALLENGE	IMPACT	SOLUTION
COLD-START EMPTY DASHBOARD	New users see blank screen with no data	Created AI onboarding questionnaire with Gemini 2.5 Flash to provide instant architecture recommendations
PROPHET CATASTROPHIC FORECASTS ON LIMITED DATA	MAPE > 1,000% with < 2 months of data	Designed a model-selection strategy by data age: Naive → Seasonal Naive → ETS (favored over Prophet for limited data)
RDS MEMORY METRIC MISMATCH	CloudWatch FreeableMemory unreliable for rightsizing	Skipped memory constraint for RDS in LP solver, using CPU-only rightsizing
CLOUDWATCH API RATE LIMITING	9 TPS limit causes pipeline crashes on real AWS	Three-layer defense: sequential processing, per-call error handling, retry with backoff

4.11 Summary

The experimental evaluation demonstrated that OptiCloud achieves all its stated objectives. The forecasting engine achieved its best overall accuracy with ETS ($\approx 11.2\%$ MAPE in walk-forward cross-validation), with Seasonal Naive most accurate at the shortest horizons; the user selects the model from the dashboard, with ETS as the recommended default based on the offline cross-validation. The

cost optimization engine identified \$590.40 in monthly savings through 19 actionable recommendations across six AWS services. The anomaly detection module identified five cost surges in the synthetic data using a global $\mu+2\sigma$ threshold. The AI recommendation module passed all 12 tests and generates structured architecture recommendations for new users. The complete test suite of 217 passing tests confirms the correctness of all system components.



CHAPTER 5

Run the Application



5.1 Introduction

This chapter provides a comprehensive walkthrough of OptiCloud's user interface and functionality. It covers system startup, the two operation modes (Demo and Real), and a detailed tour of all five dashboard pages with example use cases.

5.2 System Startup

5.2.1 Prerequisites

Before launching OptiCloud, ensure the following are installed:

- **Python 3.12 or higher**
- **pip** (Python package manager)
- **Git** (for cloning the repository)
- **SQLite3** (included with Python)
- **Optional:** AWS credentials for Real Mode (not needed for Demo Mode)

5.2.2 Installation and Launch

Step 1: Clone the repository and enter the project directory:

```
git clone <repository-url>
```

```
cd cloud-gp
```

Step 2: Create and activate a virtual environment:

```
python3 -m venv venv
```

```
source venv/bin/activate # On Windows: venv\Scripts\activate
```

Step 3: Install dependencies:

```
pip install -r requirements.txt
```

Step 4: Launch the Streamlit dashboard

```
streamlit run app.py
```

The application opens at <http://localhost:8501> with the authentication gate as the first screen.

5.2.3 Authentication Gate

The initial screen presents three options:

1. **Login** — Sign in with an existing email and password
2. **Register** — Create a new account (password hashed with PBKDF2-HMAC-SHA256)
3. **Try Demo Mode** — Access pre-loaded synthetic data without registration

For first-time exploration, click "Try Demo Mode" to load an environment preloaded with 365 days of synthetic cost history, 8 EC2 instances, 2 RDS instances, and pre-calculated optimization recommendations.

5.3 Demo Mode Walkthrough

Demo Mode logs in as demo@cis.asu.edu.eg and is intended to open the dashboard without requiring AWS credentials. (This demo account must first be seeded into the shipped database; otherwise "Try Demo Mode" fails silently.) The pre-loaded data simulates a mid-size SaaS startup with a monthly AWS bill of approximately \$1,500–\$2,500.

5.3.1 Home Page

Location: First page after authentication

Purpose: High-level overview of the user's cloud costs and key recommendations

Key Metrics Displayed:

- Total monthly cost (aggregated across all services)
- Projected monthly savings if all recommendations are implemented
- Number of cost anomalies detected in the past 30 days
- Count of available optimization recommendations (displayed alongside the savings metric rather than as a separate card)

Visualizations:

- **30-day cost trend line chart** — Shows daily costs over the trailing 30 days
- **Top 5 services by cost (pie chart)** — Displays service cost distribution
- **Top 3 recommendations (priority cards)** — Shows highest-impact recommendations with savings amounts
- **Recent anomalies (warning list)** — Lists detected cost spikes with severity levels

User Action Example: A CTO looking for quick wins would scan the top 3 recommendations and immediately see that switching 6 EC2 instances to Reserved 1-year pricing saves \$208.42/month.

5.3.2 Costs Page

Location: Sidebar navigation → "Costs"

Purpose: Detailed cost analysis by time period, service, and region

Controls:

- **Date range selector** with presets (Last 7 days, Last 30 days, Last 90 days, Custom range)
- **Export to CSV** button for sharing reports

Key Visualizations:

- **Summary metrics:** Total cost, average daily cost, minimum and maximum daily costs
- **Daily cost line chart** — Zooming and panning enabled via Plotly
- **Service breakdown bar chart** — Costs per service over selected period
- **Service breakdown pie chart** — Percentage distribution of costs
- **Stacked area chart** — Trend of each service's cost over time
- **Detailed cost table** — Daily and service-level costs, sortable and filterable

User Action Example: A DevOps engineer investigates a cost spike on March 15 by selecting the date range around that date. They see that EC2 costs spiked 40% on that day, suggesting a sudden scaling event or misconfiguration.

5.3.3 Forecasts Page

Location: Sidebar navigation → "Forecasts"

Purpose: Predict future cloud costs using a selectable time-series ML model

Controls:

- **Model selector (dropdown):** Prophet, SARIMAX, ETS, Seasonal Naive, Naive
- **Forecast horizon (dropdown):** 7, 14, 30, 60, or 90 days
- **Automatic forecast (no separate generate button)**

Workflow:

1. User selects a model from the dropdown (the default is Prophet)
2. User chooses a forecast horizon (default: 30 days)
3. The forecast runs automatically once a model and horizon are selected
4. **Loading spinner** shows that the model is training
5. Dashboard displays the forecast with upper and lower confidence bounds

Key Visualizations:

- **Forecast line chart** with shaded confidence interval band
- **Forecast data table** showing predicted costs and confidence intervals for each day
- **Model performance metrics** (MAPE, RMSE, MAE) via historical backtesting — coming soon; not yet shown
- **Multi-model comparison mode** — Optional side-by-side comparison of all models on the same forecast period

User Action Example: A Finance Manager selects the 30-day horizon and the ETS model. The forecast projects a 10% cost increase for the next month, so they budget accordingly.

5.3.4 Recommendations Page

Location: Sidebar navigation → "Recommendations"

Purpose: Browse, filter, and understand cost-saving recommendations

Controls:

- **Summary metrics:** Total recommendations, total possible monthly savings, average savings per recommendation, high-priority count
- **Multi-select filters:**
 - Service (EC2, RDS, Lambda, S3, EBS, DynamoDB, NAT Gateway, ELB)
 - Recommendation type (rightsizing, pricing plan switch, storage class switch, etc.)
 - Priority level (high, medium, low)
 - Minimum savings threshold (e.g., show only recommendations saving > \$10/month)
- **Sorting options:** By savings (descending), by priority, by service name
- **Export to CSV** button

Recommendation Card Details:

Each recommendation displays:

- Service name and resource ID
- Current configuration (e.g., "t3.large, on-demand")
- Recommended configuration (e.g., "t3.medium, on-demand")
- Current monthly cost and estimated cost
- Monthly savings and savings percentage
- Confidence level with explanation
- **Implementation guidance** (expandable section with step-by-step instructions)

User Action Example: A Platform Engineer filters to show only "high confidence" EC2 recommendations saving > \$50/month. They see the RDS rightsizing recommendation that saves \$327/month and click "View Details" to

understand why: prod-postgres is running at P95 CPU of 38%, so it can be safely moved from r5.large (\$1.009/hour) to r5.large-equivalent on Graviton (c7g.large, \$0.57/hour).

5.3.5 Settings Page

Location: Sidebar navigation → "Settings"

Purpose: User account management and AWS connection configuration

Controls:

Account Section:

- Display name (editable)
- Email address (read-only)
- Password change option (not yet implemented)
- Account creation date (not currently displayed)

AWS Connections Section:

- List of all connected AWS accounts
- For each connection:
 - Connection name
 - AWS Account ID
 - Sync status (success, failed, in progress, never)
 - Test connection button (only on the Add Account form, not per saved connection)
 - Delete connection button
- **Add New AWS Account button** with a form for:
 - Connection name
 - AWS Account ID
 - IAM Role ARN
 - External ID (optional)
 - AWS Region (default: us-east-1)

System Status Section (read-only):

- Demo Mode status
- AWS Region
- Database backend (SQLite)
- No database path, data-collection time, user count, or API quota is displayed.

User Action Example: A CTO with multiple AWS environments (production and staging) navigates to Settings and clicks "Add AWS Account". They paste the IAM Role ARN for their staging environment and click "Test Connection". The system confirms it can assume the role, and the staging connection is saved.

5.4 Real Mode Setup

For users with actual AWS accounts, the following steps enable Real Mode operation.

5.4.1 Create an IAM Role in AWS

In your AWS account, create an IAM Role with the following permissions:

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Action": [  
        "ce:GetCostAndUsage",  
        "ce:GetAnomalies",  
        "cloudwatch:GetMetricStatistics",  
        "ec2:Describe*",  
        "rds:Describe*",  
      ]  
    }  
  ]  
}
```

```

    "lambda:ListFunctions",
    "s3:ListBuckets",
    "elasticache:Describe*",
    "ecs:Describe*",
    "elasticloadbalancing:Describe*",
    "dynamodb:Describe*",
    "pricing:GetProducts",
    "sts:GetCallerIdentity"
  ],
  "Resource": "*"
}
]
}

```

The trust relationship should allow the OptiCloud system to assume this role (or use cross-account trust if deploying OptiCloud in a different AWS account).

5.4.2 Connect AWS Account via Dashboard

1. Click **Register** on the authentication gate
2. Enter email, password, and profile name
3. Click **Register** to create account
4. You are automatically logged in; navigate to **Settings**
5. Click **Add New AWS Account**
6. Paste your IAM Role ARN
7. Click **Test Connection**
8. If successful, click **Add Account**
9. The new connection is saved to OptiCloud's account list

5.4.3 Running Data Collection

When the collector is run from the command line, the CollectorRunner fetches data month by month. This initial collection takes 15–30 minutes depending on AWS API response times and data volume.

5.5 Key User Workflows

Workflow 1: New AWS User Looking for Architecture Guidance

1. Register for an account (no AWS setup needed)
2. Complete the AI onboarding questionnaire (9 questions about workload type, budget, uptime, etc.)
3. System calls Gemini 2.5 Flash with user's answers
4. Dashboard displays recommended architecture, estimated monthly cost, and detailed explanation
5. User can review the recommendation and share the details with their engineering team (an in-app bookmark or save-as-PDF option is planned for a future version)

Workflow 2: DevOps Engineer Monitoring Existing Infrastructure

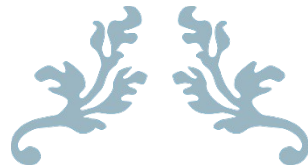
1. Log in with AWS account already connected
2. Visit **Home page** for quick status check
3. If cost is increasing unexpectedly, visit **Costs page** to pinpoint which service spiked
4. Visit **Recommendations page** to review cost-saving options
5. Implement highest-ROI recommendation (e.g., Reserved Instance purchase) via AWS Console
6. Mark recommendation as "completed" in OptiCloud (future version feature)

Workflow 3: Finance Manager Budgeting for Next Quarter

1. Log in to dashboard
 2. Visit **Forecasts page**
 3. Select 90-day horizon to project Q2 costs
 4. Use forecast to set budget caps with CFO
 5. Export forecast to CSV and share with finance team
 6. Set up monthly cost report export from **Costs page**
-

5.6 Summary

OptiCloud provides an intuitive, interactive interface accessible to both technical and non-technical users. Demo Mode is intended to enable exploration without AWS credentials, while Real Mode supports multi-account AWS environments with secure IAM role-based access. The five-page dashboard design organizes functionality logically: Home for overview, Costs for analysis, Forecasts for planning, Recommendations for action, and Settings for account management.



CHAPTER 6

Conclusion and Future Work



6.1 Project Summary

This project developed OptiCloud, an AWS Cost Intelligence System that combines machine learning, artificial intelligence, and optimization algorithms to reduce cloud waste and improve cost visibility. The system addresses two

distinct user populations: new AWS users seeking architecture guidance, and existing users needing automated cost optimization.

6.1.1 Objectives Achieved

All stated project objectives were completed:

Cost Forecasting — Implemented five forecasting models, with the system designed to guide model selection by data availability. ETS achieved the best overall accuracy (~11.2% MAPE in walk-forward cross-validation), while Seasonal Naive was most accurate at the shortest horizons (7–14 days, ~10–11% MAPE).

Anomaly Detection — Combined Z-Score and IQR methods to detect cost spikes and distributional outliers. A global $\mu+2\sigma$ threshold ($\approx \$120.38/\text{day}$; maximum z-score 6.9) flagged five cost surges in the synthetic data.

Cost Optimization — Developed a hybrid approach combining a Binary LP solver for compute rightsizing and eight heuristic rules for service-specific optimization. Identified \$590.40/month in savings from 19 recommendations on synthetic data.

AI-Based Architecture Recommendations — Integrated Google Gemini 2.5 Flash with nine-question onboarding to guide new users through architecture design without requiring AWS expertise.

User Authentication & Multi-Tenancy — Implemented PBKDF2-HMAC-SHA256 password hashing and user_id-keyed database schema for secure multi-user isolation.

Demo Mode — Pre-loaded 365 days of synthetic data based on open-source datasets, enabling immediate exploration without AWS credentials.

Unified Dashboard — Created a five-page interactive Streamlit interface with an authentication gate, account switcher, and visualizations.

6.1.2 Key Technical Achievements

- **8-component modular architecture** — Clear separation of concerns enables maintainability and future extensions
 - **SQLite as single data source** — 30-table schema with ACID transactions, WAL mode concurrency, and automatic deduplication
 - **Cross-account AWS support** — STS role assumption enables multi-account environments without storing secret keys
 - **217 passing unit and integration tests** — Ensures reliability across all modules
 - **Real-time data collection** — 11 service-specific collectors process 12 months of data with retry logic and error handling
-

6.2 Key Findings

6.2.1 Forecasting Insights

The most significant finding was that **Prophet is unsuitable for 30-day cost forecasting regardless of data availability**. Even with 1.5 years of training data, ETS outperformed Prophet for this horizon. This informed offline guidance on model choice by horizon, based on the cross-validation results:

- **Seasonal Naive** was most accurate at the shortest horizons (7–14 days, MAPE ~10–11%)
- **ETS** was the most accurate at longer horizons (21–90 days) and best overall (~11.2% MAPE)
- **Prophet** was among the least accurate across all horizons (MAPE ~14–22%) and is not recommended

6.2.2 Anomaly Detection Robustness

The union approach (flagging if either Z-Score or IQR detects an anomaly) maximized recall, with the design intent that flagged cost anomalies be

excluded from forecasting training data. While this admits slightly higher false positives, the trade-off is acceptable because:

- Exclusion errors are more harmful than inclusion errors
- False positives on real-world data are rare given the 30-day rolling window and 1.5× IQR multiplier

6.2.3 Optimization Coverage

The hybrid LP + heuristic approach covered all major AWS services:

- **Compute-heavy optimization** (EC2, RDS) → Binary LP solver with P95 CPU + 1.3× headroom
- **Service-specific optimization** (Lambda, S3, DynamoDB, NAT, ELB) → Eight targeted heuristic rules

The P95 metric with 30% headroom proved effective, avoiding over-engineering for rare spikes while catching legitimate underprovisioning.

6.3 Impact and Value

OptiCloud delivers measurable value across three dimensions:

6.3.1 Cost Reduction

On the synthetic demo environment, the system identified **\$590.40/month in potential savings** (19 recommendations). Against the \$2,189.48/month AWS bill across 42 resources, this represents a **27% cost reduction** potential. As an illustrative projection applied across a portfolio of 100 companies, this yields:

- **\$59,040 projected total monthly savings** ($100 \times \$590.40$)
- **\$708,480 projected annual savings** ($59,040 \times 12$)

6.3.2 Time Savings

Manual optimization requires deep AWS expertise. OptiCloud automates this:

- **Automatic cost forecasting** eliminates 2–4 hours of manual spreadsheet work per month

- **Recommendation generation** replaces 8–16 hours of manual instance review per month
- **Dashboard updates** eliminate weekly status meetings to discuss cost trends

For a team of 5 engineers, this totals **an estimated 140–240 hours/year** saved per customer.

6.3.3 Accessibility

OptiCloud broadens access to AWS cost optimization:

- **New users** can design appropriate architectures through AI guidance without hiring expensive consultants
 - **Small teams** can implement recommendations without deep AWS expertise
 - **Multi-account organizations** can centralize cost management across all accounts from a single interface
-

6.4 Limitations and Trade-offs

6.4.1 Current Limitations

1. **AWS-only implementation** — Current system targets AWS exclusively. Extending to Azure and GCP would require API changes per cloud provider.
2. **ETS model performance plateau** — ETS MAPE plateaus at ~11% for 30-day forecasts; further improvement would require feature engineering or ensemble methods.
3. **RDS memory metrics unreliable** — CloudWatch reports raw bytes rather than utilization percentage, making RDS memory-based rightsizing impossible. CPU-only rightsizing is a reasonable compromise but leaves potential savings on the table.

4. **Manual recommendation approval** — Recommendations are informational only; users must apply them manually via AWS Console. Automation of actual changes would require additional IAM permissions and safety mechanisms.
5. **Single-zone demo environment** — Synthetic data does not include multi-AZ failover costs or regional arbitrage optimization opportunities.

6.4.2 Known Trade-offs

- **LP solver + Heuristics vs. end-to-end ML:** Chose interpretability and explainability over potential marginal accuracy gains from end-to-end neural networks.
- **Z-Score + IQR union vs. advanced methods:** Chose recall (catching all anomalies) over precision (minimizing false positives) because the design treats excluding anomalies from training data as critical.
- **Temperature 0.3 for Gemini:** Chose determinism over creativity to ensure consistent architecture recommendations, sacrificing some flexibility in handling edge cases.

6.5 Future Work

6.5.1 Short-term Enhancements (1–3 months)

1. **Recommendation approval workflow**
 - Add checkboxes to mark recommendations as "implemented" or "dismissed"
 - Track actual savings realized from implemented recommendations
 - Calculate accuracy of forecasts vs. actual costs as feedback loop
2. **Fine-tuned LLM for architecture recommendations**
 - Current: Google Gemini 2.5 Flash (closed source)

- o Future: Fine-tune an open-source LLM (Llama 2, Mistral) on OptiCloud's questionnaire + AWS best practices corpus
- o Benefit: Reduced API costs, better AWS-specific guidance, offline capability

3. **Export-as-code features**

- o **Terraform export:** Convert recommendations to Terraform modules for Infrastructure-as-Code deployment
- o **CloudFormation templates:** Generate CFN templates for Reserved Instance purchases
- o **Automation:** Streamline application of recommendations from OptiCloud directly to AWS

4. **Email notifications**

- o Cost spike alerts: "Your EC2 costs increased 40% on March 15"
- o Recommendation reminders: "High-priority recommendation available: save \$327/month on RDS"
- o Forecast notifications: "Q2 projected cost is \$8,500; budget approval needed"

6.5.2 Medium-term Extensions (3–6 months)

1. **Multi-cloud support**

- o Extend to Azure and Google Cloud Platform
- o Unified cost comparison dashboard: "Same workload costs \$X on AWS, \$Y on Azure, \$Z on GCP"
- o Generic optimization algorithms that work across providers

2. **Advanced forecasting**

- o **Ensemble models:** Weighted combination of Prophet, ETS, and LSTM for hybrid predictions

- **Exogenous variable support:** Incorporate traffic forecasts, seasonal business events, and capacity planning signals
- **Uncertainty quantification:** Bayesian methods to provide confidence intervals reflecting both model uncertainty and data uncertainty

3. **Real-time cost tracking**

- Currently: Daily cost updates from AWS Cost Explorer
- Future: Real-time meter from CloudWatch with 1-hour latency
- Enable intra-day cost anomaly detection and immediate alerts

4. **Savings validation loop**

- Track which recommendations were implemented and when
- Compare forecasted savings vs. actual realized savings
- Retrain models using the feedback signal to improve recommendations

6.5.3 Long-term Vision (6–12 months)

1. **Autonomous cost management**

- Graduated trust model: Start with read-only recommendations → Low-risk automated actions → Higher-risk automation with approval gates
- Example: Automatically purchase Reserved Instances when confidence > 95% and savings > threshold
- Require executive approval for termination of "idle" resources (safety catch)

2. **Industry-specific benchmarking**

- o Collect anonymized cost data across customers in same vertical (SaaS, E-commerce, Media, etc.)
- o Enable peer comparison: "Your AWS costs are 15% higher than industry median for your company size"
- o Industry best practices database: "How other SaaS startups optimized their infrastructure"

3. Predictive capacity planning

- o Forecast resource needs 3–6 months ahead based on business metrics (user growth, revenue, hiring plans)
- o Recommend Reserved Instance quantities and expiration timing
- o Optimize commitment discounts (Savings Plans, Compute Savings Plans)

4. FinOps maturity scoring

- o Assess organization's cloud financial maturity: "You're at Level 2 (Managed)/Level 5 (Autonomous)"
- o Recommend next-step improvements in cost governance, chargeback models, and optimization practices
- o Integrate with ITFM (IT Financial Management) tools

6.6 Conclusion

OptiCloud demonstrates that intelligent automation can reduce cloud waste and improve cost visibility. By combining machine learning forecasting, heuristic optimization, and AI-based architecture guidance, the system makes cloud cost optimization accessible to organizations of all sizes and expertise levels.

The key insight—that different algorithms excel at different problems (Seasonal Naive for the shortest horizons, ETS for medium-to-long horizons; LP for compute, heuristics for service-specific)—enabled a modular design that outperforms single-model approaches.

As cloud computing becomes more critical to business operations, the ability to optimize costs while maintaining performance will be increasingly valuable. OptiCloud provides a foundation for this optimization, with clear pathways for extension to multi-cloud environments, autonomous cost management, and industry-specific insights.

References

- Shen, S. et al. "Statistical Characterization of Business-Critical Workloads Hosted in Cloud Datacenters." *IEEE/ACM CCGrid*, 2015.
- Lavin, A. & Ahmad, S. "Evaluating Real-time Anomaly Detection Algorithms — the Numanta Anomaly Benchmark." *IEEE ICMLA*, 2015.
- Cortez, P. et al. "Resource Central: Understanding and Predicting Workloads for Improved Resource Management in Large Cloud Platforms." *ACM SOSP*, 2017.
- Taylor, S.J. & Letham, B. "Forecasting at Scale." *The American Statistician*, 2018.

- Liu, G. et al. "A Comprehensive Survey of Public Cloud Datasets." *ACM Computing Surveys*, 2025.
- Flexera. "State of the Cloud Report." 2024.
- Gartner. "Cloud Cost Optimization Strategies." 2023.
- Hyndman, R.J. & Athanasopoulos, G. "Forecasting: Principles and Practice." OTexts, 2021.
- AWS. "AWS API Documentation." <https://docs.aws.amazon.com>
- Google AI. "Google Gemini API Documentation." <https://developers.google.com/generative-ai>
- Facebook. "Prophet: Time Series Forecasting." <https://facebook.github.io/prophet/>
- Streamlit. "Streamlit Documentation." <https://docs.streamlit.io>
- Bitbrains GWA-T-12. "Grid Workloads Archive." <http://gwa.ewi.tudelft.nl/datasets/gwa-t-12-bitbrains>
- NAB. "Numenta Anomaly Benchmark." <https://github.com/numenta/NAB>
- Kaggle. "AWS Pricing Dataset." <https://www.kaggle.com/datasets/justsahil/aws-pricing-dataset>